Meme Caption Generation Using LSTM and Transformers: A Comparison

Mohammed Arsh
Department of Computational Linguistics
Heidelberg University
Heidelberg, Germany
mohammed.arsh@stud.uni-heidelberg.de

Abstract

Using the *memes900k* dataset introduced by DeepHumor, I trained and compared different image captioning models. I used ResNet-50 and CLIP ViT-B/32 as frozen visual encoders, combined with two types of decoders, LSTM and Transformer. I evaluated the different encoder-decoder combinations using BLEU and perplexity, with a shared greedy decoding strategy to ensure a consistent comparison. Code: https://github.com/MohammeddArsh/memeGen.

1 Introduction

Image captioning is the task of generating a natural-language description of an image. Meme captioning, however, is more challenging because the meaning of a meme often goes beyond what is visibly present in the image. A single meme template can be used with many different captions, depending on the situation, emotion, or metaphor being expressed.

Generating meme captions therefore requires more than recognising objects in an image. It also requires learning the different ways an image can be associated with text. For this reason, the quality and variety of the training data play an important role. A dataset containing many captions for the same image can help a model learn the different ways that image may be used.

This project uses the *memes900k* dataset, which contains 300 meme templates and 900,000 captions, to investigate meme caption generation. Following the general approach of DeepHumor, I train models to learn relationships between meme images and their associated captions. I compare ResNet-50 and CLIP ViT-B/32 as image encoders, combined with LSTM and Transformer decoders, to examine how these choices affect caption generation.

The aim of this project is to understand which model components have the greatest influence on the quality of generated meme captions. Rather than explicitly teaching the models to understand humour or metaphor, I investigate how well they can learn caption patterns from existing examples.

2 Background and Related Work

Image captioning models often combine neural networks that extract image features with models that generate text. Early approaches used CNNs and RNNs (Vinyals et al., 2015), while later methods introduced attention mechanisms (Xu et al., 2015) and Transformers (Vaswani et al., 2017). CLIP uses large-scale image and text data to learn visual representations (Radford et al., 2021).

Meme captioning is more challenging because a single image can have many captions, often expressing humour or metaphor. DeepHumor (Borovik et al., 2020) explored this task using the *memes900k* dataset, containing 300 meme templates and 900,000 captions.

Building on this work, my project compares ResNet-50 and CLIP encoders with LSTM and Transformer decoders. I evaluate the generated captions using BLEU (Papineni et al., 2002) and measure text prediction using perplexity, applying the same greedy decoding method across all models.

3 Method

3.1 Corpus

Table 1: Official caption-level split of *memes900k*.

Split	Rows	Per template
Train	750,000	2,500
Validation	75,000	250
Test	75,000	250

The *memes900k* dataset contains 300 meme templates and 900,000 captions. I used the official split (Table 1), with 750,000 training, 75,000 validation,

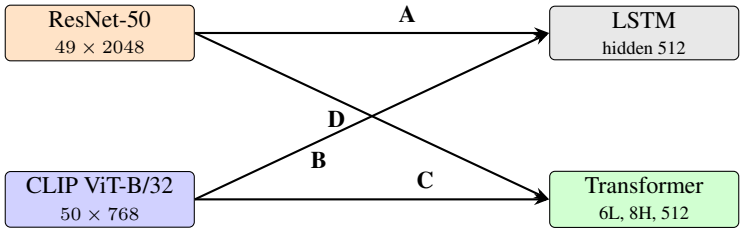


Figure 1: All encoders are frozen and their features precomputed. Runs: A = ResNet-50+LSTM, B = CLIP+LSTM, C = CLIP+Transformer, D = ResNet-50+Transformer; E = C with embedding 768.

and 75,000 test captions. Since the same templates appear in all three splits, the evaluation measures how well models generate new captions for familiar images, rather than captions for unseen templates. Captions are tokenized using the GPT-2 byte-pair encoding (BPE) vocabulary (Radford et al., 2019), which contains 50,257 tokens. Each caption is formatted as <bos> TOP <sep> BOTTOM <eos> and limited to 64 tokens. Shorter sequences are padded on the right using <pad>, which is excluded from the loss calculation. All model configurations use the same vocabulary, allowing their perplexity scores to be compared consistently within this study.

Content warning. *memes900k* is scraped from user-generated internet memes and contains offensive material, including racial and ethnic slurs, profanity, and other vulgar or hateful language and imagery. I neither filtered nor sanitized it; models trained on it including ours, can reproduce such content.

3.2 Architectures

Encoders. Used two frozen image encoders: ResNet-50 (He et al., 2016), which produces 49 visual features of size 2048, and CLIP ViT-B/32 (Radford et al., 2021), which produces 50 visual features of size 768. These features are extracted before training.

Decoders. I compare an LSTM (Hochreiter and Schmidhuber, 1997) with two layers and a Transformer (Vaswani et al., 2017) with six layers. The LSTM uses pooled image features to initialise its hidden state, while the Transformer attends to the image features when generating captions.

I combined both encoders with both decoders (Figure 1). These four configurations are labelled A–D. Configuration E uses CLIP with a Transformer and a larger embedding size of 768.

3.3 Training and Budgets

All models use the same training settings, AdamW with a learning rate of 3×10^{-4}, 500 warmup steps, weight decay of 10^{-4}, gradient accumulation of 4, batch size 32, and random seed 42. Training uses a subset of 200,000 caption rows. With a batch size of 32, one epoch consists of 6,250 steps.

I used two training budgets. **Tier 1** trains each model for 1 epoch (6,250 steps), while **Tier 2** trains for 3 epochs (18,750 steps), matching the longer training setup used in DeepHumor. Hyperparameters are first tested for 6,000 steps, changing one setting at a time and selecting the best configuration based on validation loss. The selected configuration is then tested using the full training budget. The results section (Section 4) describes this process, and Figure 2 shows the training and validation loss curves.

3.4 Metrics

Perplexity: $\text{ppl} = \exp(\overline{\text{NLL}})$, calculated under teacher forcing across all 75,000 test rows. Perplexity is comparable only within this project's GPT-2 BPE vocabulary.

BLEU: Corpus BLEU is calculated using `sacrebleu` on a fixed, seeded sample of 5,000 test rows, with greedy decoding (Papineni et al., 2002; Post, 2018). BLEU is used to compare the generated captions across all model configurations.

4 Results

Table 2: Tier 1 results (1 epoch, official test split).

Run	Encoder	Decoder	BLEU	Ppl
A	ResNet-50	LSTM	2.13	255.5
B	CLIP	LSTM	8.92	259.7
C	CLIP	Transformer	**14.57**	71.5
D	ResNet-50	Transformer	14.50	72.7
E	CLIP	Transformer (768)	14.36	**68.4**

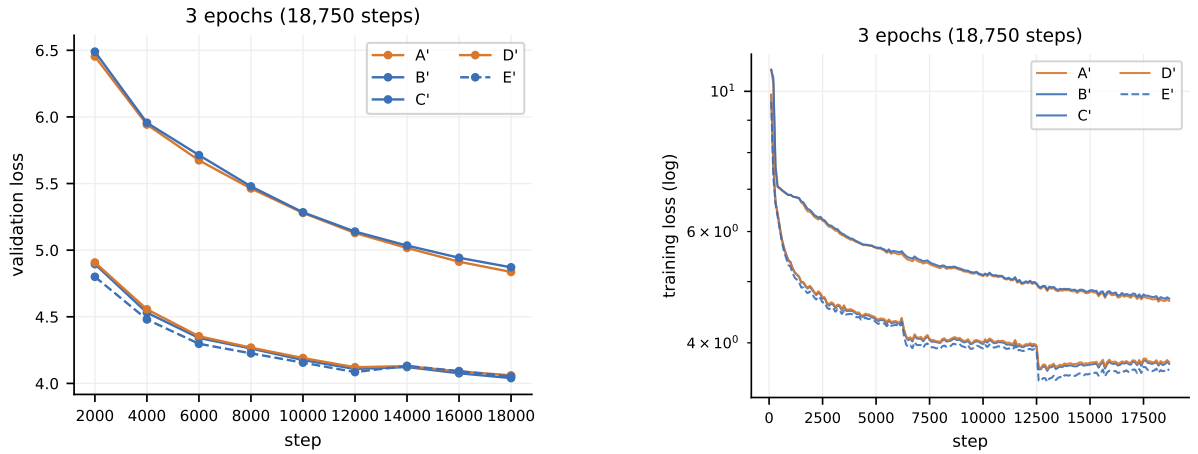


Figure 2: Training dynamics on the 3-epoch (18,750-step) tier. Left: validation loss; right: training loss (log scale). The Transformer runs converge faster and lower.

4.1 Tier 1 — 1 epoch (6,250 steps)

Table 2 reports Tier 1. The decoder dominates, using Transformer over LSTM gives +5.65 BLEU (B→C) and +12.37 (A→D). The encoder matters only when the decoder is weak (+6.79 for A→B) and is flat behind a Transformer (C→D = −0.07). A larger embedding (E) improves perplexity but not BLEU.

Table 3: Tier 2 results (3 epochs, official test split).

Run	Encoder	Decoder	BLEU	Ppl
A′	ResNet-50	LSTM	13.36	115.9
B′	CLIP	LSTM	12.16	120.7
C′	CLIP	Transformer	14.38	**54.7**
D′	ResNet-50	Transformer	14.63	55.5
E′	CLIP	Transformer (768)	**14.66**	55.2

4.2 Tier 2 — 3 epochs (18,750 steps)

Table 3 reports Tier 2. The Transformer runs form a tight cluster (14.38–14.66), and both LSTM runs remain behind (12.16–13.36) even though they improve substantially over Tier 1. Perplexity roughly halves relative to Tier 1 while BLEU is essentially unchanged (Figure 4).

4.3 Hyperparameter screening

Figure 3 ranks the six screening configurations by validation loss at 6,000 steps. Increasing the embedding to 768 is the only change that beats the base configuration; the remaining knobs are within noise of it.

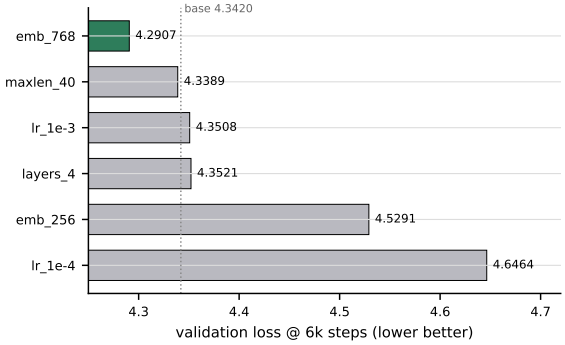


Figure 3: One-knob-at-a-time screening at 6,000 steps. Validation loss (lower is better); the embed-768 variant (highlighted) is the winner and was confirmed at full budget.

5 Discussion

The decoder has the greatest impact. Across both training budgets, the Transformer generally performs better than the LSTM. CLIP provides a clear improvement over ResNet-50 when paired with the LSTM after one epoch, but this difference changes after longer training. With the Transformer, BLEU scores remain similar across both encoders and embedding sizes.

Perplexity does not necessarily reflect caption quality. The model with a larger embedding size was selected based on validation loss, but its BLEU score was similar to those of the other Transformer models. This shows that better text prediction does not always lead to better generated captions. Therefore, BLEU provides useful additional information when comparing models.

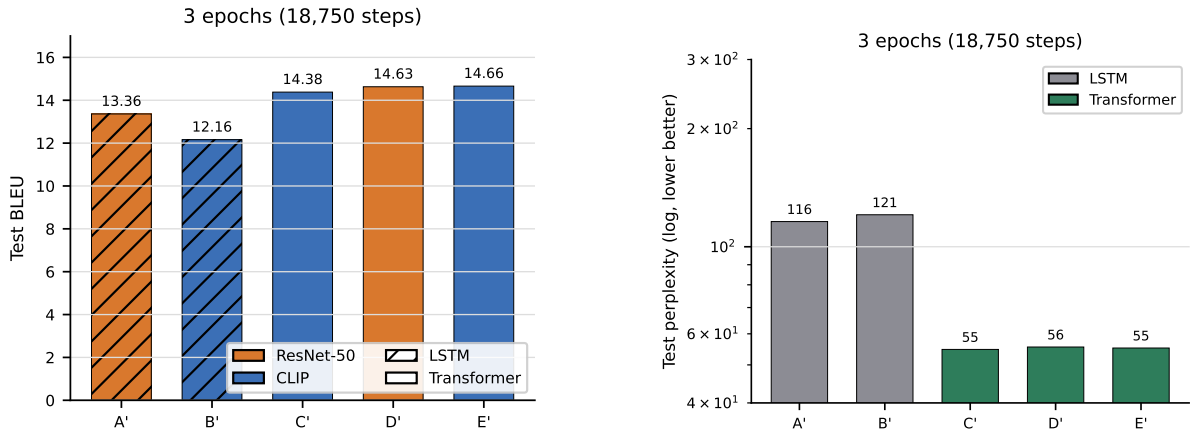


Figure 4: Test metrics by run on the 3-epoch (18,750-step) tier. Left: BLEU (bars coloured by encoder, hatched by decoder). Right: perplexity (log scale, lower is better). The Transformer runs dominate.

The effect of the encoder depends on the decoder. After one epoch, the choice of encoder has a noticeable effect when using the LSTM, but very little effect with the Transformer. After three epochs, the differences between encoder-decoder combinations become smaller. These results suggest that the decoder architecture has a greater influence on caption generation than the choice of visual encoder in our experiments.

6 Conclusion

The results on *memes900k* show that the choice of decoder has the greatest effect on caption generation. Transformer models achieve around 14.5 BLEU, while LSTM models score between 12 and 13 after longer training. With a Transformer decoder, changing the encoder or embedding size has little effect. The results also show that lower perplexity does not necessarily lead to better captions, as perplexity decreases with further training while BLEU scores remain similar.

Qualitative examples. Figure 5 compares reference (G) and generated (P) captions for three models trained for three epochs. The generated captions are generally fluent but often differ from the references. This is expected because a single meme template can have many valid captions, making exact matches difficult.

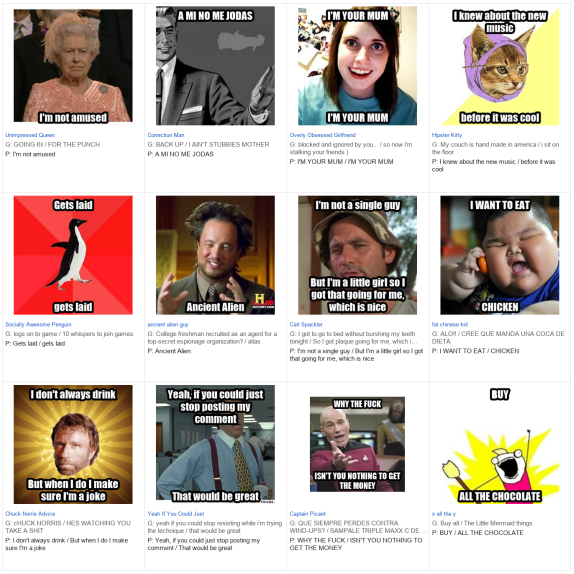
7 Interactive Demo

I developed an interactive demo using a Google Colab notebook that launches a GRADIO web app. The app generates captions and displays them on meme images. Users can select one of the 300 meme templates or upload their own image. The generated caption is displayed on the image using the standard Impact meme font. The app also allows users to choose a decoding strategy, adjust the sampling temperature, and generate multiple caption variations side by side.

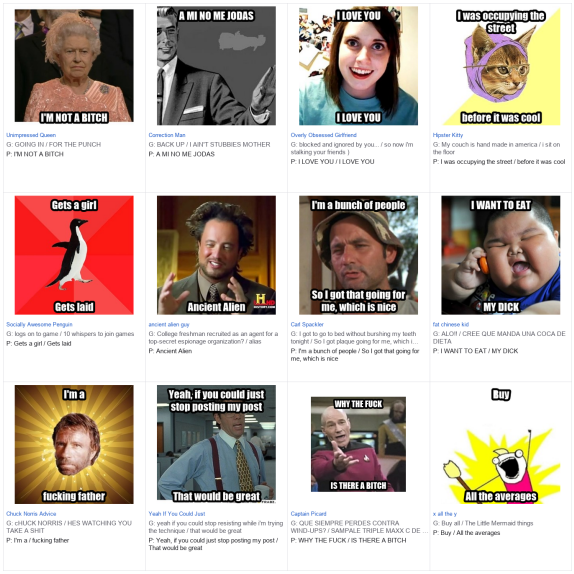
The demo supports two decoding strategies. *Greedy decoding* selects the most likely token at each step, producing the same caption for the same input. I use this method to evaluate all models in Section 4 because it provides consistent and reproducible results. In contrast, *sampling* selects tokens from a probability distribution, allowing the model to generate different captions for the same image. I use a temperature of $\tau = 0.9$ by default and apply top-k filtering with $k = 50$ to limit the selection to the most likely tokens. Lower temperatures make the output more predictable, while higher temperatures encourage greater variety. This makes sampling useful for exploring different caption ideas, although its randomness makes it less suitable for consistent evaluation.

The demo also accepts images that are not part of the training dataset. Since the visual encoder can extract features from different images, the model can attempt to generate captions for uploaded images as well as the original meme templates. However, these examples are intended for demonstration only. My evaluation uses templates that appear in the training data, so the results do not establish how well the model generalises to completely unseen templates.

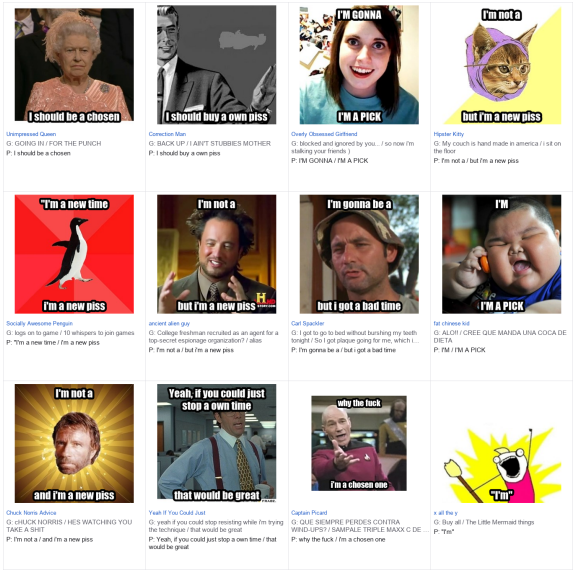
For the demo, I selected the CLIP+Transformer model with an embedding size of 768 (Run E′). This model performed best during hyperparameter



best run (CLIP+Transformer, 3 ep, emb 768)



ResNet-50+Transformer (3 ep)



CLIP+LSTM (3 ep)

Figure 5: Gold (G) vs. generated (P) captions for three 3-epoch runs. Generated captions are fluent but often diverge from the reference, which is expected given the many valid captions per template and directly explains the low absolute BLEU.

screening and achieved the highest BLEU score (14.66) and lowest perplexity (55.2) among the Tier 2 models. I therefore chose it to demonstrate the caption generation capabilities of the trained models.

Limitations

1. **Seen templates only:** The training, validation, and test sets share meme templates, so the results do not show how well the models perform on unseen templates.

2. **BLEU has limitations:** Many different captions can be valid for the same meme. BLEU may give a low score to a reasonable caption simply because its wording differs from the reference.

3. **Perplexity comparison:** Our models use the GPT-2 BPE tokenizer, while DeepHumor uses a vocabulary built from the dataset. Their perplexity scores are therefore not directly comparable.

4. **Training and generation differ:** The models are trained using reference captions but generate captions one token at a time during evaluation. This difference remains unresolved in our experiments.

5. **Limited training setup:** The visual encoders remain frozen, and training uses a 200,000-row subset rather than the full training set.

Ethical Statement

The *memes900k* dataset is used for non-commercial academic research under its intended use; no human annotators were involved and no new personal data were collected. All models are trained on frozen, publicly available pretrained features, so the compute footprint is modest (Apple Silicon MPS, tens of GPU-hours). I reused the open-source DeepHumor codebase in line with its license and cite it accordingly. Meme generation can reproduce or amplify offensive stereotypes present in the source data; I therefore report aggregate metrics only.

AI Use Statement

Generative AI tools like Open Code and chatGPT were used for limited assistance with code and LaTeX formatting. The research design, experiments, analysis, and the writing of this manuscript were conceived and carried out by the human author, who remains fully responsible for the content, accuracy, and originality of this submission.

References

Ilya Borovik, Bulat Khabibullin, Vladislav Kniazev, Zakhar Pichugin, and Oluwafemi Olaleke. 2020. Deephumor: Image-based meme generation using deep learning. https://github.com/ilya16/deephumor. Final project, Skoltech Deep Learning course.

Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. 2016. Deep residual learning for image recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 770–778.

Sepp Hochreiter and Jürgen Schmidhuber. 1997. Long short-term memory. *Neural Computation*, 9(8):1735–1780.

Kishore Papineni, Salim Roukos, Todd Ward, and Wei-Jing Zhu. 2002. Bleu: a method for automatic evaluation of machine translation. In *Proceedings of the 40th Annual Meeting of the Association for Computational Linguistics*, pages 311–318.

Matt Post. 2018. A call for clarity in reporting bleu scores. In *Proceedings of the Third Conference on Machine Translation: Research Papers*, pages 186–191.

Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, et al. 2021. Learning transferable visual models from natural language supervision. In *International Conference on Machine Learning*, pages 8748–8763. PMLR.

Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. 2019. Language models are unsupervised multitask learners. *OpenAI blog*, 1(8):9.

Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. *Advances in Neural Information Processing Systems*, 30.

Oriol Vinyals, Alexander Toshev, Samy Bengio, and Dumitru Erhan. 2015. Show and tell: A neural image caption generator. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 3156–3164.

Kelvin Xu, Jimmy Ba, Ryan Kiros, Kyunghyun Cho, Aaron Courville, Ruslan Salakhudinov, Rich Zemel, and Yoshua Bengio. 2015. Show, attend and tell: Neural image caption generation with visual attention. In *International Conference on Machine Learning*, pages 2048–2057. PMLR.